

Reviewer Pack — Spectral Gap Lower Bound on Tseitin Resolution Length

Entry 14b9250ca476 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 14:26:45 UTC

Spectral Gap Lower Bound on Tseitin Resolution Length

Entry ID: 14b9250ca476 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 14:26:45 UTC

1. Conjecture

Field A (mathematical branch): SPECTRAL_GRAPH_THEORY **Field B** (complexity object): RESOLUTION_LENGTH

Statement:

For a d -regular graph G with algebraic connectivity λ_2 , the resolution length of its Tseitin formula is at least $2^{\Omega(\lambda_2)}$. Non-expanders (with $\lambda_2 = o(1)$) have resolution length subexponential in λ_2 .

Rationale (proposer's reasoning):

Spectral gap (λ_2) quantifies expansion properties of graphs. Expanders (large λ_2) are known to require exponential resolution for Tseitin formulas. This conjecture links λ_2 directly to resolution complexity, offering a new graph invariant for hardness characterization.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9b1146352b29a699

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
---------	---------	------------	------------------	------------------

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        n = len(A)
        for i in range(n):
            max_row = i + max(range(i, n), key=lambda j: abs(A[j][i]))
            A[i], A[max_row] = A[max_row], A[i]
            factor = 1 / A[i][i]
            for j in range(i, n):
                A[i][j] *= factor
            for k in range(n):
                if k != i:
                    factor = A[k][i]
                    for j in range(i, n):
                        A[k][j] -= factor * A[i][j]
        return [row[i] for row in A]

    def matrix_multiplication(A, B):
        m, n, p = len(A), len(B[0]), len(B)
        C = [[0] * n for _ in range(m)]
        for i in range(m):
            for j in range(n):
                for k in range(p):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def laplacian_eigenvalues(G, d):
        n = len(G)
        L = [[0] * n for _ in range(n)]
        for i in range(n):
            L[i][i] = d - sum(G[i])
            for j in range(i + 1, n):
                if G[i][j]:
                    L[i][j] = L[j][i] = -G[i][j]
        eigenvalues = gaussian_elimination(L)
        return sorted(eigenvalues)
```

```

def tseitin_resolution_length(G, d):
    n = len(G)
    lambda_2 = laplacian_eigenvalues(G, d)[1]
    if lambda_2 < 1e-6:
        return 0
    return 2 ** (math.log(lambda_2) / math.log(2))

def generate_d_regular_graph(n, d):
    G = [[0] * n for _ in range(n)]
    degree = [0] * n
    edges_added = set()
    while any(d != sum(G[i]) for i in range(n)):
        u = random.randint(0, n - 1)
        v = random.randint(0, n - 1)
        if u == v or G[u][v] or (u, v) in edges_added:
            continue
        G[u][v] = G[v][u] = 1
        degree[u] += 1
        degree[v] += 1
        edges_added.add((u, v))
    return G

n_values = [5, 10, 15, 20, 30, 40]
total_length = 0
instances_tested = 0
conjecture_holds = True
counterexample = ""

for n in n_values:
    for _ in range(5): # Aim for at least 30 instances per seed
        d = random.randint(2, min(n - 1, 4))
        G = generate_d_regular_graph(n, d)
        length = tseitin_resolution_length(G, d)
        total_length += length
        instances_tested += 1

        if lambda_2 < 1e-6:
            expected_length = 0
        else:
            expected_length = 2 ** (math.log(lambda_2) / math.log(2))

        if abs(length - expected_length) > 1e-4:
            conjecture_holds = False
            counterexample = f"Graph with n={n}, d={d} has unexpected resolution length {length}"

    return {
        "metric_name": "Tseitin Resolution Length",
        "metric_value": total_length / instances_tested,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 8)]

```

```

results = []

for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")

mean_length = sum(result["metric_value"] for result in results) / len(results)
std_length = math.sqrt(sum((result["metric_value"] - mean_length)**2 for result in results) / len(results))
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_length} std={std_length} support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing evaluation of support fraction or counterexamples. | next: Run test with extended timeout and higher resource allocation

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	91144
2	propose	ollama_remote	qwen3:8b	0	0	63985
3	preregistration	ollama_remote	qwen3:8b	0	0	25731
4	novelty	ollama_remote	qwen3:8b	0	0	20600
5	novelty	ollama_remote	qwen3:8b	0	0	12971
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	41795
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12961
8	judge	ollama_remote	qwen3:8b	0	0	14825

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 284010 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/14b9250ca476.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/14b9250ca476.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/14b9250ca476.tar.gz (if generated)